

Six Claude Code Repos Worth Exploring — A Research Briefing

Prepared for: Evan Grobbelaar (ECS Community) **Source:** "6 Claude Code GitHub Repos That Change Everything" — Nuno Tavares / Automated Marketer (YouTube, May 2026) **Context:** Compiled by Tony to share with the ECS community, with integration notes tied to the kind of work we already do — agent workflows, persistent memory, MCP-driven automation (e.g. the ThinkMarkets trading setup), and the Tony-trading knowledge repo.

How to read this

Each repo below is grouped by what it actually does, not the marketing. For every one you get: the canonical GitHub link, what it is, maturity/adoption signals, and a short "where this fits for us" note so the community can see concrete integration angles rather than just a tool list.

A recurring theme across all six: the value of Claude Code isn't the model, it's the **harness** around it — skills, hooks, persistent memory, and multi-agent coordination. That maps directly onto problems we keep hitting: agents forgetting context between sessions, inconsistent behaviour, and the manual glue work of re-explaining our setup every time.

1. Superpowers — obra/superpowers

Link: <https://github.com/obra/superpowers>

What it is: An agentic skills framework and software-development methodology by Jesse Vincent (Prime Radiant). It installs a session-start hook that loads a "using-superpowers" bootstrap, then auto-triggers skills based on what you're doing. Instead of jumping straight to code, the agent steps back, teases a spec out of the conversation, shows it back in digestible chunks, then produces an implementation plan before writing anything.

Notable skills: test-driven-development (write failing tests first), systematic-debugging (a 4-phase root-cause process), verification-before-completion, subagent-driven-development (two-stage review: spec compliance, then code quality), and finishing-a-development-branch (verify tests, then merge/PR/discard cleanly).

Maturity / signals: Mature and opinionated. Works across many harnesses (Claude Code, Codex, Cursor, Gemini CLI, Copilot CLI, and more). Zero-dependency by design. The maintainers run adversarial eval testing and famously reject ~94% of PRs — a sign of tight quality control. Optional, opt-out telemetry only.

Where this fits for us: This is the cleanest answer to "make the agent stop guessing and start with a spec." For the trading-context work, the brainstorming-before-code behaviour and verification-before-completion skills would directly reduce the kind of silent-assumption errors that have been costly. It's also a good reference architecture for how a SessionStart hook can inject our own standing rules (the PING routine, risk rules) automatically.

2. Everything Claude Code (ECC) — affaan-m/everything-claude-code

Link: <https://github.com/affaan-m/everything-claude-code>

What it is: The big one. A production configuration framework born from an Anthropic hackathon win (Affaan Mustafa, Sept 2025), now one of the most-starred repos on GitHub (160K+ stars, 25K+ forks by mid-2026). It packages dozens of specialized agents, 180+ skills, slash commands, rules, 20+ hooks, and MCP server configs into one installable, version-controlled system that works across Claude Code, Cursor, Codex, and OpenCode from a single repo (AGENTS.md at root is read by all four).

Standout features: - **Hook-based security enforcement** — PreToolUse hooks block `--no-verify` git flags, detect secrets in prompts (patterns like `sk-`, `ghp_`, `AKIA`), and stop agents from editing linter configs instead of fixing the actual code. - **AgentShield** — a security scanner with 1,282 tests and 102 static-analysis rules; an `--opus` flag runs a red-team/blue-team/auditor pipeline against your own agent config. - **Instincts / self-learning loop** — a 5-layer observer loop watches sessions, spots recurring patterns, and codifies successful behaviours into persistent "instincts" managed via slash commands (`/instinct-status`, `/instinct-evolve`, `/instinct-prune`). - Selective-install architecture and runtime hook controls (`ECC_HOOK_PROFILE`, `ECC_DISABLED_HOOKS`).

Business model: Core repo is free and MIT-licensed. A paid GitHub App ("ECC Tools") adds private-repo analysis and auto-generated skill files (Pro ~\$19/seat/mo). Worth knowing the free core is the substantive part.

Caveats: Despite the star count it isn't universally praised — it's a large surface area, and some of the "instincts" tooling is heavier than small projects need.

Where this fits for us: The two features most relevant to ECS are the **secret-detection hooks** and **AgentShield**. Given our past concerns about plaintext tokens and auto-execute patterns in the Tony-trading workflow, ECC's hook that blocks `sk-` / `ghp_` / `AKIA` patterns from leaking into prompts is exactly the guardrail we discussed wanting. The persistent-memory / instincts pattern is also the closest off-the-shelf analogue to the master-context-document approach — worth studying even if we don't adopt it wholesale.

3. Ruflo (formerly Claude Flow) — `ruvnet/ruflo`

Link: <https://github.com/ruvnet/ruflo>

What it is: An agent "meta-harness" by Reuven Cohen — an orchestration layer that turns Claude Code into a multi-agent swarm. 100+ specialized agents that self-organize into swarms (mesh/hierarchical/ring/star topologies), self-learning memory, federation across machines, and an MCP server exposing a very large tool surface. The v3.5 rebrand (Feb 2026) moved the policy engine, embeddings, and proof system from TypeScript to Rust/WASM. (~59K stars; npm package and CLI are still `claude-flow`.)

Maturity / signals — read this carefully: This is the most "buyer beware" entry. It's actively developed with near-daily releases, but a widely-circulated independent audit (May 2026) tested the tool hands-on and concluded that a large share of the 300+ MCP tools are stubs, state records with no executor, or cosmetic output that wastes tokens — with only a few components (real memory, real subagent spawning via Claude's native Agent tool) doing meaningful work. Most performance claims are self-reported. The maintainer engaged constructively with the audit and shipped fixes, so it's improving, but treat headline numbers skeptically.

Where this fits for us: Genuine multi-agent orchestration is appealing for anything that fans out into parallel sub-tasks (e.g. researching macro news across several instruments at once before a trade). But given the audit findings, I'd recommend **trialling it in a sandbox before depending on it**, and watching token consumption closely — runaway token use is a real risk and maps badly onto a frequent-ping workflow. The audit's own recommendation (lean on a few proven MCP servers plus native tools rather than the full swarm) is sound advice for us.

4. Open Design — `nexu-io/open-design`

Link: <https://github.com/nexu-io/open-design>

What it is: A local-first, open-source alternative to Anthropic's Claude Design (Apache-2.0). It turns a coding-agent CLI you already have into a design engine: 250+ skills, 140+ portable `DESIGN.md` brand systems (Linear, Stripe, Vercel, Notion, Apple, Anthropic, etc.), and a sandboxed iframe preview. It generates web/desktop/mobile prototypes, dashboards, slide decks, images, and video, then exports to HTML/PDF/PPTX/MP4. Everything lands as real files in your project, version-controllable, not locked in a vendor cloud.

How it integrates: Auto-detects 16+ coding-agent CLIs on your PATH (Claude Code, Codex, Cursor, Gemini, OpenCode, etc.). Ships a stdio MCP server (`od mcp`) so any MCP-aware agent in another repo can read your design files — tokens CSS, JSX components, entry HTML — as a queryable structured API. BYOK for the model provider.

Maturity / signals: Very young but exploding (57K+ stars within ~8 weeks of launch; ~v0.9 as of June 2026). The architecture is sound and the loop runs end-to-end, but the project self-describes as early — fine for exploration, not yet a guaranteed-stable foundation for client deliverables.

Where this fits for us: Most relevant if ECS produces any front-facing material — dashboards, one-pagers, decks. The "every artifact is a real file you can git-diff and hand to a dev" model is the standout. For a trading dashboard concept (live P&L, gold-silver ratio, open positions) this could generate a real editable HTML artifact rather than a throwaway mockup. The MCP-server-exposing-design-files angle is also a clean pattern to study for how we might expose our own repo files to agents.

5. Obsidian Skills — `kepano/obsidian-skills`

Link: <https://github.com/kepano/obsidian-skills>

What it is: Agent Skills for working with Obsidian, authored by Steph Ango (kepano, Obsidian's CEO). They follow the Agent Skills specification, so they work with any skills-compatible agent — Claude Code, Codex, OpenCode. They teach the agent to use the Obsidian CLI and open formats: Markdown, Bases, and JSON Canvas. Install via `/plugin marketplace add kepano/obsidian-skills` then `/plugin install obsidian@obsidian-skills`, or via `npm install obsidian@obsidian-skills`.

Related projects worth knowing (the ecosystem here is busy, and the video's "Obsidian" segment overlaps with these): `AgriciDaniel/claude-obsidian` and `qhuang20/obsidian-skills` both build a self-organizing "second brain" using Karpathy's LLM-Wiki pattern — drop in sources, the agent extracts entities/concepts, cross-links them, and files them into a structured vault that compounds over time. These lean toward knowledge management; kepano's is the canonical, format-focused skill set.

Maturity / signals: kepano's repo is authoritative (first-party from Obsidian's founder) and tracks the official Agent Skills spec. The LLM-Wiki community repos are newer and more experimental but conceptually rich.

Where this fits for us: This is the most direct match to the **Tony-trading knowledge repo** problem. The LLM-Wiki pattern — a persistent, interlinked Markdown knowledge graph the agent builds and queries instead of re-reading raw docs every session — is essentially a more structured version of the master context document. If ECS members keep trading notes, research, or playbooks in Markdown, this turns them into something an agent can ingest, link, and answer from with citations to specific pages. Strong candidate for a community pilot.

6. Karpathy Skills Plugin — `multica-ai/andrej-karpathy-skills`

Links: - Canonical: <https://github.com/multica-ai/andrej-karpathy-skills> - Original (redirects here):

What it is: A short, human-readable `CLAUDE.md` (~70 lines) by Forrest Chang, written the day after Andrej Karpathy's Jan 2026 post on how LLMs fail at coding. It distills four behavioural principles to counter the most common failure modes: 1. **Think Before Coding** — state assumptions, present alternative interpretations, ask when ambiguous, don't pick silently. 2. **Simplicity First** — minimum code that solves the problem; no speculative features or premature abstractions. 3. **Surgical Changes** — touch only what's required; match existing style; don't refactor working code. 4. **Goal-Driven Execution** — turn vague imperatives into verifiable success criteria so the model can loop toward a measurable target.

The Multica version goes beyond the single file: it ships as a proper Claude Code plugin (`.claude-plugin/`), a structured skill (`skills/karpathy-guidelines/SKILL.md`), and a Cursor rule. Install via `/plugin marketplace add` then `/plugin install`.

Maturity / signals: Extremely popular (combined 220K+ stars across the personal and org repos by mid-2026) and dead simple — it's a prompt, not a hard constraint, so it raises the behavioural floor without guaranteeing it. **Security note:** because `CLAUDE.md` files can be weaponized (a malicious one in a cloned repo can instruct the agent to exfiltrate secrets), only pull this from the two official repos above, and read the 70 lines before installing. A Feb 2026 Snyk audit found 13% of agent-skills packages had critical flaws — relevant to all the skill repos in this briefing, not just this one.

Where this fits for us: This is the lowest-effort, highest-leverage adoption of the six — a 30-second install that directly attacks the "why did it also change that / it assumed the wrong thing" problem. Given that silent wrong assumptions have been our single most expensive recurring mistake pattern, the "Think Before Coding" and "Surgical Changes" principles are worth adopting as a baseline `CLAUDE.md` for any ECS repo, including Tony-trading. Layer project-specific rules after the Karpathy content (later rules override).

Quick comparison

Repo	Category	Best for	Maturity	Effort to adopt
Superpowers (<code>obra/superpowers</code>)	Skills + methodology	Spec-first, disciplined dev loops	High	Low-Medium
Everything Claude Code (<code>affaan-m/everything-claude-code</code>)	Full config framework	Security hooks, persistent memory, scale	High	Medium-High
Ruflo (<code>ruvnet/ruflo</code>)	Multi-agent orchestration	Parallel swarm tasks	Active but contested	High (sandbox first)
Open Design (<code>nexu-io/open-design</code>)	Design artifacts	Dashboards, decks, prototypes as real files	Early (v0.9)	Medium
Obsidian Skills (<code>kepano/obsidian-skills</code>)	Knowledge management	Compounding Markdown knowledge graph	Authoritative	Low-Medium
Karpathy Skills (<code>multica-ai/andrej-karpathy-skills</code>)	Behavioural guardrails	Stopping bad assumptions/over-engineering	Popular, simple	Very low

Suggested order for the ECS community

1. **Start with Karpathy Skills** — trivial install, immediate behavioural improvement, sets a baseline everyone can share.
2. **Then Superpowers** — adds the spec-first, verify-before-done discipline on top.
3. **Pilot Obsidian Skills (LLM-Wiki pattern)** for the shared knowledge / trading-notes problem.
4. **Mine Everything Claude Code** for its security hooks and AgentShield even if you don't adopt the whole framework — the secret-detection guardrails are directly relevant to our token-handling concerns.
5. **Explore Open Design** if/when there's a front-facing dashboard or deck to build.
6. **Sandbox Ruflo** last and cautiously — verify it earns its token cost before relying on it.

A cross-cutting security note

Four of these six install skills or `CLAUDE.md / SKILL.md` files that the agent will follow as standing instructions. The same audits cited above (Adversa AI, LayerX, Snyk) show this is a real attack surface. Practical rules for the community: install only from the official repos linked here; read `CLAUDE.md / SKILL.md` files before enabling them; and keep secret-detection hooks (ECC's, or equivalent) in place so nothing exfiltrates credentials regardless of which skills are loaded.